ITS za razvoj veština pisanja održivog koda

I. Problem

Inženjeri softvera treba da pišu *kvalitetan* kod koji ispunjava funkcionalne zahteve. Važna karakteristika *kvaliteta* koda je *održivost*. Održivost određuje lakoću razumevanja, izmene, proširivanja i testiranja softverskih komponenti [1]. Kod niske održivosti uvećava cenu razvoja softvera i loše utiče na ostale aspekte njegovog kvaliteta, poput bezbednosti, performansi i pouzdanosti [2][3]. Stoga industrija i akademija ističu da je važno da inženjeri softvera nauče da pišu održiv kod [4][5][6][7].

Postoji mnoštvo heuristika za pisanje održivog koda [8][9][10]. Mnoge su neprecizni opisi podložni subjektivnim interpretacijama [11] ili su previše tesno vezane za programske jezike, tehnološke konvencije ili arhitekturalne šablone [12]. Posledično, veštine pisanja održivog koda se stiču kroz izradu velikog broja zadataka uz podršku eksperta koji daje povratne informacije. Međutim, postoji oskudica eksperata koji bi mogli predavati ovo gradivo [13].

Inteligentni Tutoring Sistemi (*Intelligent Tutoring Systems*, ITS) bi mogli umanjiti ovaj problem simulacijom ljudskog instruktora. Ovaj rad predstavlja ITS specijalizovan za učenje pisanje održivog koda. Glavna komponenta ovog ITS-a je zadužena da prima kod od učenika i vraća personalizovane povratne informacije za ispravku tog koda.

Razvijeni ITS može da pomogne učenicima da savladaju osnove veštine pisanja održivog koda kroz manje izazove. Zahvaljujući tome, instruktori mogu posvetiti više vremena radu sa učenicima na sofisticiranijim problemima. ITS takođe podržava instruktore u otkrivanju miskoncepcija u znanju učenika.

II. Teorijske osnove

Poglavlje II.A opisuje domen pisanja održivog koda. Kroz ovo poglavlje ilustrujemo kako ne postoji precizna definicija održivog koda, te se veština pisanja održivog koda mora učiti kroz primere. Poglavlje II.B analizira ciljeve interesnih grupa i na osnovu njih definiše zahteve koje rešenje treba da ispuni kako bi doprinelo ispunjenju tih ciljeva.

A. *Održivost koda*

NAPOMENA: poglavlje nije napisano, ali je iznad istaknut njegov cilj. Okvirno, organizacija poglavlja bi bila: (1) definicija pojma održivosti koda, (2) konkretan primer problema održivosti koda (npr., *Long Method*). Primer bi pokazivao kako ne postoje precizne (merljive) heuristike koje jednoznačno definišu funkciju kao „dugačku“.

B. *Zahtevi koje mora da ispuni edukativni alat za razvoj veštine pisanja održivog koda*

Zahtevi koje treba da ispuni edukativni alat za razvoj veštine pisanje održivog koda su izvedeni iz našeg iskustva kao instruktora i empirijskih studija koje su istraživale potrebe instruktora za analitikama učenja u učionici [2][5]. Tabela II prikazuje zahteve koje rešenje treba da ispuni i objašnjava kako oni doprinose ispunjenju ciljeva interesnih grupa koje će koristiti rešenje.

C. *Teorijske osnove rešenja koje ispunjava izvedene zahteve*

ITS je tip tehnologije učenja koji modeluje principe efektivne instrukcije (*model instruktora*) da adaptira edukativni sadržaj (*model domena*) specifičnostima učenika (*model učenika*) [2]. ITS-ov *korisnički interfejs* definiše instrumente (*learning instrument*) koji prezentuju edukativni sadržaj i rukuju interakcijom učenika sa sadržajem [2].

Cilj ITS-a je da razvije temeljno znanje učenika u najkraćem mogućem roku [3]. ITS prati interakcije studenata sa edukativnim sadržajem kako bi podržao svoju prilagodljivost i omogućio analitiku učenja [4]. Na osnovu prikupljenih događaja, instruktori mogu da identifikuju učenike kojima je potrebna pomoć, uobičajene miskoncepcije,

Tabela II Zahtevi alata za učenje veštine pisanja održivog koda

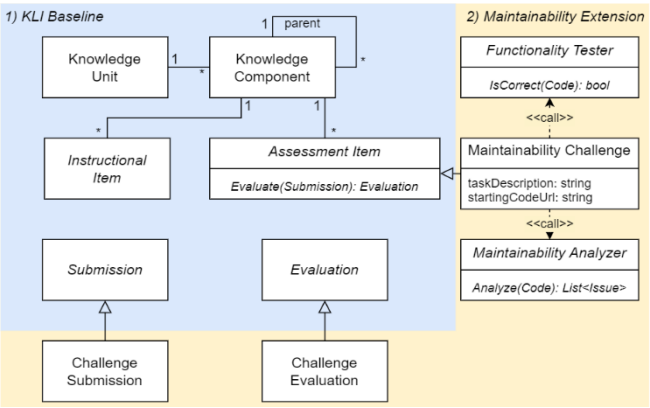
Cilj interesne grupe	Zahtev	Argument kako zahtev doprinosi cilju
Studenti softverskog inženjerstva žele efektivno da savladaju veštinu pisanja održivog koda kako bi unapredili svoje karijere	1. Alat treba da simulira situacije iz realnog sveta koje zahtevaju upotrebu veštine pisanja održivog koda	Realistično okruženje povećava angažovanost u učenju i rezultate učenja, pomažući učenicima da razviju veštine potrebne za dato realistično okruženje [1]. Na primer, pitanje sa višestrukim izborom je manje realistično od programskog zadatka koji se rešava u integrisanom razvojnom okruženju
	2. Alat treba da automatski identifikuje nedostatke u znanju svakog studenta	Rešenje treba da automatski identifikuje nedostatke u znanju svakog studenta kako bi se omogućilo automatsko prilagođavanje instrukcije
	3. Alat treba da se automatski prilagodi nedostacima u znanju studenta odabirom adekvatnih zadataka i povratnih informacija	Rešenje treba da se adaptira nedostacima u znanju svakog studenta kako bi se optimizovali dobici u učenju i time efikasno razvile veštine učenika.
Instruktori teže donošenju odluka zasnovanih na podacima kako bi poboljšali iskustvo učenja za sebe i studente	4. Alat treba da nadgleda i prikazuje progres svakog studenta	Pomoću pregleda aktivnosti svakog učenika, instruktor može da usmeri svoju pažnju na studente kojima treba pomoć, dok alat obučava ostale studente.
	5. Alat treba da prati interakcije studenata sa edukativnim sadržajem da bi podržao analizu kvaliteta sadržaja	Pomoću ove funkcionalnosti, instruktori mogu da analiziraju studentske odgovore na zadatke, vreme provedeno u svakom segmentu ili šablone u učenju. Na taj način instruktori razumeju kojim delovima procesa učenja i edukativnih materijala je potrebno unapređenje. Specifično za održivost koda, instruktor može da analizira predat kod da bi utvrdio kršenje principa, preporuka i heuristika i adekvatno razvio model veština pisanja održivog koda.

preterano izazovne zadatke i edukativni sadržaj koji je neophodno unaprediti [5].

Postoji više pristupa dizajnu ITS [6]. Pristup koji se ističe po svojoj zrelosti i nivou empirijske verifikacije je *Knowledge-Learning-Instruction* (KLI) radni okvir [3]. KLI definiše koncepte koji modeluju znanje, učenje i instrukciju (slika 1, deo). *Instruktivni elementi* (*instructional items*) su varijacije u okruženju učenja koje uzrokuju porast znanja učenika. Na primer, ovde spadaju edukativni materijali prezentovani u formi videa ili teksta koji izlažu gradivo. *Instruktivni elementi* razvijaju nove i rafiniraju postojeće veštine (*knowledge components*) koje učenici moraju da primene da bi rešili probleme iz domena. Domenski problemi su modelovani kroz *zadatke* (*assessment items*). Primeri *zadataka* mogu biti zadaci koji zahtevaju kraći odgovor na pitanje ili pisanje koda koji ispunjava određene zahteve. *Zadaci evaluiraju* (*Evaluate*) učenički *odgovor* (*Submission*) da bi utvrdili stepen razvoja *veština* učenika i vratili adekvatne povratne informacije.

III. REŠENJE

Slika 1 predstavlja domenski model rešenja. Osnova domenskog modela je *Knowledge-Learning-Instruction* (KLI) radni okvir predstavljen u poglavlju III.A. Nad ovom osnovom je izgrađena *Maintainabillity Challege* komponenta ITS-a (poglavlje III.B) koja je fokus ovog rada. Ova komponenta je specijalizovana za proveru koliko je učenika vešt u pisanju održivog koda.



Slika 1 Domenski model inteligentnog tutoring sistema za razvoj veštine pisanja održivog koda.

A. Knowledge-Learning-Instruction (KLI) radni okvir

Koncepti KLI radnog okvira su prikazani na segmentu 1) slike 1.

Svaki tip zadatka se izvodi iz entiteta *AssesmentItem* i implementira funkciju *Evaluate*. Svaki zadatak sadrži logiku koja:

1. prima odgovor (*Submission*)
2. proverava tačnost odgovora i, prema tome, određuje odgovarajuće povratne informacije (*feedback*)
3. vraća tačnost odgovora i povratne informacije kao evaluaciju (*Evaluation*).

Da bismo podržali specijalizovanu *Maintainability Challenge* komponentu, iz *Submission* i *Evaluation* klasa smo izveli klase *ChallengeSubmission* i *ChallengeEvaluation*. Ove klase enkapsuliraju specifičan ulaz koga očekujemo od učenika i odgovarajući izlaz koji vraća sistem.

Svaki specijalizovan zadatak zahteva i instrument (*learning instrument*) sa kojim učenik interaguje. Instrument je deo ITS-ovog korisničkog interfejsa.

B. Maintainabillity Challege komponenta

Izazov održivosti (*Maintainabillity Challege*) je specijalizovan tip zadatka za razvoj veština analize održivosti i refaktorisanja koda. Prikazan je u okviru segmenta 2) slike 1. Učenici predaju *rešenje izazova* (*challenge submission*), koje *izazov održivosti* evaluira da bi proizveo evaluaciju izazova (*challenge evaluation*).

Svaki izazov opisuje *zadatak* koji učenik treba da uradi. Primeri zadataka su:

- implementacija funkcije,
- promena ponašanja postojeće funkcije
- refaktorisanje funkcije da bi se njena održivost povećala, pri čemu se mora voditi računa da njena funkcionalnost ne bude izmenjena.

Izazov može da pruži *početni kod.* Početni kod može biti prazna funkcija, klasa sa delimično implementiranom logikom ili veliki projekat sa mnogo paketa i klasa. Ova varijabilnost omogućava instruktorima da podese obim zadataka – od ilustrativnog izazova za početnike, do realnih primera softverskih projekata koji mogu biti analizirani po pitanju održivosti i refaktorisani.

Pošto izazov zahteva da učenici kreiraju ili modifikuju kod, uveli smo tester funkcionalnosti (*functionality tester*) koji određuje da li je kod funkcionalno korektan. Tester funkcionalnosti nad kodom pušta automatizovane testove. Predstavlja značajan element procene održivosti koda, jer je kod koji nije funkcionalno korektan neadekvatan, bez obzira na svoj kvalitet [6].

Kod koji je učenik predao se analizira od strane *Maintainability Analyzer* komponente (slika 2):

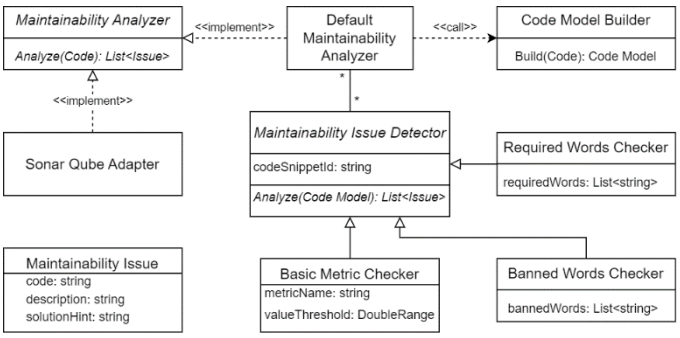
1. Kod se procesira u model koda koji je pogodniji za analizu održivosti koda. Logika za procesiranje koda je enkapsulirana u (*Code Model Builder*) klasi. Logika procesiranja koda u model koda je tesno vezana za programski jezik, dok je model koda donekle nezavisan[1] od programskog jezika.
2. Na model koda se primenjuje jedan ili više detektora problema održivosti (*Maintainability Issue Detector*). Svaki detektor problema održivosti je primenljiv na ceo predati kod ili na deo predatog koda, definisan kroz *codeSnippetID* identifikator. Ovaj identifikator se poklapa sa potpisom klase (*class namespace* ii me klase) ili punim potpisom metode (koji obuhvata potpis klase i metode).

Dizajnirali smo tri tipa detektora problema održivosti:

1. *Basic Metric Checker* koji poredi vrednosti metrika koda sa predefinisanim pragovima. Primer ovakvog detektora je određivanje da li sve funkcije u kodu imaju između pet i 15 linija koda (metrika *Lines Of Code*, LOC)

[1] Slični jezici (na primer, *Java* i C#) mogu biti konvertovani u isti model koda. Međutim, konvertovanje bitno različitih jezika (na primer, *JavaScript* i C) u zajednički model je upitno, pogotovo što se problemi održivosti u tom slučaju mogu značajno razlikovati.

2. *Banned Word Checker* osigurava da predefinisani skup neadekvatnih reči (*noise words* [5]) nije prisutan u predatom kodu. Na primer, ovakav detektor problema vrši proveru da li sva imena u predatom kodu ne sadrže generičke reči poput „the“, „data“, „info“, „int“, „list“ ili „var“.
3. *Required Word Checker* osigurava da predati kod sadrži imena iz predefinisanog skupa reči. Na primer, ovakav detektor bi proverio da li imena u kodu sadrže „DataRange“ i „Overlaps“.



Slika 2 Konceptualni model analize održivosti koda (*Maintainability Analyzer*).

NAPOMENA: poglavlje nije dovršeno. Primer ukazuje kako da opisujete svoj sistem. Obratite pažnju da tekst ne ponavlja imena i odnose klasa prikazanih na slikama, već daje dodatne informacije o tome kako je sistem dizajniran.

IV. Evaluacija rešenja

Da bismo ocenili korisnost i unapredili dizajn alata sproveli smo četiri empirijske evaluacije sa različitim interesnim grupama (Tabela III).

A. Grupna diskusija sa softverskim inženjerima

Grupna diskusija (*focus group discussion*) [7] sa softverskim inženjerima se sastojala iz dva dela. Tokom prva dva sata, softverski inženjeri su interagovali sa razvijenim *izazovima održivosti* (poglavlje III.B). Vođe eksperimenta su posmatrale tok interakcije korisnika, odgovarale na postavljena pitanja i beležile probleme sa upotrebljivošću alata (*usability issues*). Nakon eksperimenta, usledila je grupna diskusija fokusirana na predloge za unapređenje korisničkog interfejsa i edukativnog sadržaja.

Svi softverski inženjeri su pohvalili razvijene izazove održivosti i ITS. Posebno su pohvalili integraciju izazova u integrisano radno okruženje (*integrated development environment*), koja omogućava upotrebu njima poznatih alata za refaktorisanje. Opšti zaključak softverskih inženjera je bio da ITS adekvatno simulira realistično radno okruženje, pogotovo u slučaju težih zadataka.

B. Kontrolisani eksperiment sa studentima

Kontrolisani eksperiment (*controlled experiment*) sproveden je podelom studenata u dve grupe prateći radnomizovan blok dizajn (*randomized block design*). Kontrolna grupa (*control group*, $N_c = 18$) je koristila *Learning Management System*, tradicionalno korišćen alat za prezentaciju edukativnog sadržaja. Eksperimentalna grupa (*experiment group*, $N_c = 33$) je koristila razvijeni ITS. Za obe grupe smo izmerili:

- *Studentsko angažovanje.* Instruktori su posmatrali obe grupe učesnika tokom eksperimenta i beležili svoja zapažanja. Studenti su nakon eksperimenta popunili anketu u kojoj su izvršili samo-procenu svog nivoa agažovanja.
- *Stečeno znanje.* Studenti su nakon eksperimenta demonstrirali stečeno znanje na testu.
- *Robusnost stečenog znanja.* Studenti su nedelju dana nakon eksperimenta ponovo uradili isti test znanja.
- *Upotrebljivost ITS*-a i *doživljaj korisnosti*. Studenti iz eksperimentalne grupe su popunili anketu u kojoj su ocenili ove indikatore.

NAPOMENA: poglavlje nije dovršeno. U poglavlju bi bili opisani: (1) procedura eksperimenta (na primer, kako su studenti podeljeni u grupe, koja konkretna pitanja su korišćena za procenu izmerenih indikatora), (2) rezultati (prosečne ocene na anketi, statistički testovi, diskusija uočenih rezultata).

C. Grupna diskusija sa instruktorima

Pet univerzitetskih instruktora je koristilo razvijeni ITS i učestvovalo u pratećoj diskusiji. Primarni cilj diskusije je bio validacija dizajna kontrola za učenike (zahtevi 2 i 3) i instruktore (zahtevi 4 i 5). Sekundarni cilj je bio podizanje svesti o mogućnostima razvijenog alata i priprema alata i edukativnog sadržaja za terenski eksperiment.

Instruktori su dva sata interagovali sa ITS-om u ulozi učenika, nakon čega su pola sata posmatrali instruktorske kontrole. Zapisali su svaku sprovedenu akciju (iz liste definisanih akcija) zajedno sa vremenom njenog sprovođenja

Tabela III Empirijske studije sprovedene u cilju evaluacije razvijenog ITS-a

Cilj evaluacije	Metod evaluacije	Učesnici u eksperimentu	Broj učesnika
Validacija zahteva 1	Grupna diskusija	Softverski inženjeri zaposleni u industriji srednjeg radnog staža (2-5 godina radnog iskustva)	5
Validacija da li razvijeni ITS može da zameni tradicionalno korišćen *Learning Management System*	Kontrolisani eksperiment	Studenti treće godine osnovnih akademskih studija	51
Validacija dizajna kontrola za instruktore (zahtevi 4 i 5)	Grupna diskusija	Instruktori softverskog inženjerstva zaposleni na fakultetu srednjeg radnog staža (2-8 godina radnog iskustva)	5
Validacija dizajna kontrola za učenike (zahtevi 2 i 3)	Terenski eksperiment	Studenti druge godine osnovnih akademskih studija	59

u dnevnik aktivnosti. Kreirani dnevnik aktivnosti je iskorišćen za verifikaciju da sistem radi prema specifikaciji i korektno beleži akcije. Instruktori su takođe dokumentovali probleme koje su uočili i predloge za unapređenje alata i edukativnog sadržaja.

Konačni korak evaluacije je bila diskusija u kojoj je tim instruktora razmenio svoja iskustva i ideje za unapređenje. Prikupili smo 17 ideja za unapređenje sadržaja i 33 ideje za unapređenje alata. Većina (86%) su bili manji problemi, poput gramatičkih grešaka ili loše poravnatih elementa korisničkog interfejsa.

Instruktori su se složili da alat podstiče angažovanje i može biti koristan za prenos znanja i učenje u domenu pisanja održivog koda. Složili su se da ponuđene kontrole ispunjavaju zahteve i omogućavaju postizanje ciljeva interesnih grupa dokumentovanih u tabeli II.

D. Terenski eksperiment sa studentima

Terenski eksperiment (*field study*) [8] je sproveden u okviru kursa „Uvod u softversko inženjerstvo“. Na kursu su studenti upotrebljavali ITS da bi stekli veštinu pisanja održivog koda. Eksperiment je trajao je četiri nedelje.

Studenti su tokom eksperimenta imali stalan pristup ITS-u, pri čemu su najintenzivnije interagovali sa ITS-om tokom laboratorijskih vežbi. Asistenti su nadgledali studentske interakcije sa ITS-om tokom vežbi i kroz praćenje instruktorskih kontrola. Na kraju svake nedelje, studenti su popunjavali opcionu kratku anketu kroz koju su komentarisali (1) alat i (2) edukativni sadržaj. Nakon eksperimenta, sprovedena je grupna diskusija i završna anketa. Terenski eksperiment završen je izradom projekta na kome su studenti morali demonstrirati stečene veštine.

Pitanja na završnoj anketa dizajnirana su da identifikuju negativne efekte koje ITS ima na studente. Razmatrani su negativni efekti iz kataloga prikazanog u radu [9]. Pitanja predstavljaju tvrdnje za koje student na Likertovoj skali izražava nivo slaganja (1 – potpuno neslaganje, 5- potpuno slaganje). Tvrdnje su prikazane u tabeli IV.

Određena Likertova pitanja su praćena otvorenim pitanjima, kroz koja su studenti mogli da elaboriraju svoje odgovore u formi slobodnog teksta. Za analizu odgovora primenjena je tehnika kodiranja [10]. Dva autora rada su nezavisno izvršila kodiranje odgovora i razrešila neslaganja kroz diskusiju.

TABELA IV PITANJA NA ZAVRŠNOJ ANKETI, KLASIFIKOVANA U KATEGORIJE NEGATIVNIH EFEKATA KOJE EDUKATIVNI SISTEM MOŽE IMATI NA STUDENTE

Završno pitanje na anketi	**Negativan efekat na studenta [9]**
Korisnički interfejs ITS-a je intuitivan i lak za korišćenje.	Nepoverenje
ITS je pouzdan alat za učenje	
Edukativni sadržaj je jasan, dobro razvijen i visokog kvaliteta.	
Moja motivacija tokom sesija učenja je bila visoka.	Nezainteresovanost
Fokusi i angažovanje tokom sesija učenja su mi bili visoki.	
Bilo bi poželjno koristitsi ITS i na drugim kursevima.	
Instrukcioni sadržaj me je pripremio za zadatke.	Frustracija
Povratne informacije i *hint*-ovi su bili od pomoći.	
Naučio/la sam puno kroz ITS.	Neefektivno učenje
Primenio/la sam stečeno znanje na projektnom zadatku.	
Brzina kojom sam prolazio/la kroz edukativni sadržaj je bila spora.	Neefikasno učenje

LITERATURA

[1] R. Weinhandl, T. Houghton, and Z. Lavicza, “A case study on learning basic logical competencies when utilising technologies and real-world objects,” *Education and Information Technologies*, vol. 26, no. 1, pp. 639-653, 2021. DOI: 10.1007/s10639-020-10282-5

[2] E. Mousavinasab, N. Zarifsanaiey, S. R. Niakan Kalhori, M. Rakhshan, L. Keikha, and M. Ghazi Saeedi, “Intelligent tutoring systems: a systematic review of characteristics, applications, and evaluation methods,” *Interactive Learning Environments*, vol. 29, no. 1, pp. 142-163, 2021. DOI: 10.1080/10494820.2018.1558257

[3] K. R. Koedinger, A. T. Corbett, and C. Perfetti, “The Knowledge-Learning-Instruction framework: Bridging the science-practice chasm to enhance robust student learning,” *Cognitive science*, vol. 36, no. 5, pp. 757-798, 2012. DOI: 10.1111/j.1551-6709.2012.01245.x

[4] K. Mangaroska, and M. Giannakos, “Learning analytics for learning design: A systematic literature review of analytics-driven design to enhance learning,” *IEEE Transactions on Learning Technologies*, vol. 12, no. 4, pp. 516-534, 2018. DOI: 10.1109/TLT.2018.2868673

[5] K. Holstein, B. M. McLaren, and V. Aleven, “Intelligent tutors as teachers' aides: exploring teacher needs for real-time analytics in blended classrooms,” in *Proceedings of the seventh international learning analytics & knowledge conference*, Mar. 2017, pp. 257-266. DOI: 10.1145/3027385.3027451

[6] D. Dermeval, R. Paiva, I. I. Bittencourt, J. Vassileva, D. Borges, “Authoring tools for designing intelligent tutoring systems: a systematic review of the literature,” *International Journal of Artificial Intelligence in Education*, vol. 28, no. 3, pp. 336-384, 2018. DOI: 10.1007/s40593-017-0157-9

[7] J. F. Templeton, *The focus group: A strategic guide to organizing, conducting and analyzing the focus group interview*. Probus Publishing Company, 1994.

[8] J. Singer, S. E. Sim, and T. C. Lethbridge, “Software engineering data collection for field studies,” in *Guide to Advanced Empirical Software Engineering*, pp. 9-34, Springer, London, 2008. DOI: 10.1007/978-1-84800-044-5_1

[9] R. Pelánek, and T. Effenberger, “Improving Learning Environments: Avoiding Stupidity Perspective,” *IEEE Transactions on Learning Technologies*, vol. 15, no. 1, pp. 64-77, Feb. 2022. DOI: 10.1109/TLT.2022.3154936

[10] C. B. Seaman, “Qualitative methods in empirical studies of software engineering,” *IEEE Transactions on software engineering*, vol. 25, no. 4, pp. 557-572, 1999. DOI: 10.1109/32.799955